

ITCS 6169/8169 - Computer Vision

Assignment 1: The CNN Challenge

Due: September 25, 2026, 11:59 PM EDT

1 The Challenge

Imagine that you have just joined the computer vision team at *Company XYZ* as an Applied Research Scientist (PhD students) or Research Intern (Masters students). Your first task sounds simple:

Build the best image classifier you can from a small dataset.

You are given only **2,400 training images** belonging to **16 classes**. The dataset is available here:

[Download Dataset](#)

Your objective is to obtain the highest possible classification accuracy while developing a solution that is technically sound, reproducible, and computationally reasonable. There is deliberately no step-by-step recipe. You will need to make decisions about architecture, augmentation, optimization, regularization, validation, and training strategy. In this course, we will begin directly with training convolutional neural networks.

2 Learning Objective

The objective of this assignment is **not simply to obtain a high accuracy number**. By completing the assignment, you should demonstrate that you can:

- design and train a modern CNN-based image recognition system;
- reason about training when labeled data are limited;
- diagnose underfitting, overfitting, and optimization problems;
- conduct controlled experiments rather than randomly changing hyperparameters;
- use modern AI coding tools effectively while retaining ownership of the scientific and engineering decisions;
- maintain a reproducible research codebase using GitHub.

3 CNN Rules

The primary model in this assignment must be based on a **convolutional neural network**. You may use architectures such as ResNet, ResNeXt, DenseNet, EfficientNet, ConvNeXt, MobileNet, or a CNN architecture designed by you. You may modify the architecture, loss function, augmentation pipeline, optimizer, learning-rate schedule, regularization strategy, or training procedure. **Pretrained CNNs are allowed**, but you must clearly state what was pretrained, on which dataset, and which parameters were fine-tuned. For this assignment, the following may **not** be used as the primary classifier:

- CLIP or other vision-language models;
- DINO/DINOv2 or similar foundation-model embeddings;
- Vision Transformers used as the primary architecture;
- multimodal large language models or VLM APIs for classifying the test images.

We will study these models later in the course. Assignment 1 is intentionally about understanding CNN training.

4 AI Coding Tools: Required

It is 2026. Modern engineers and researchers routinely work with AI coding assistants. Therefore, **using an AI coding tool is required for this assignment**. Examples include ChatGPT, Codex, GitHub Copilot, Claude Code, Gemini, Cursor, or comparable tools. However,

AI is your pair programmer, not your research scientist.

You may ask AI to generate boilerplate, implement an augmentation, debug an error, refactor code, explain PyTorch functionality, create plotting utilities, or suggest experiments. You are responsible for deciding **what experiments should be run, why they should be run, whether the generated code is correct, and what conclusions can be drawn from the results**.

You must be able to explain every important component of your final system. Statements such as “the AI suggested this architecture” or “ChatGPT gave me these hyperparameters” are **not scientific justification**.

Required AI Reflection

Create a section called `AI_USAGE.md` in your GitHub repository containing:

- which AI coding tool(s) you used;
- 3–5 representative examples of how AI assisted you;
- at least one example where the AI produced an incorrect, ineffective, or questionable suggestion;
- how you verified or modified the AI-generated solution;
- one important experimental or architectural decision that **you** made rather than simply accepting the AI recommendation.

You do **not** need to provide every prompt you used. The objective is to demonstrate thoughtful use of AI rather than a transcript of your interaction with it.

5 The Experimental Process

Treat this assignment like a small research project. Begin with a reasonable CNN baseline. Once the baseline is working, identify its weaknesses and improve it through controlled experimentation. Possible questions include:

- Is the model overfitting?
- Would stronger augmentation help?
- Does model capacity matter?

- Does ImageNet initialization help substantially?
- Which optimizer or learning-rate schedule works best?
- Does label smoothing improve generalization?
- Does weight decay matter?
- Are some classes significantly more difficult than others?
- Does increasing image resolution help enough to justify the additional computation?
- Can a smaller model perform almost as well as a much larger model?

You are **not expected to test everything**. Good experimental reasoning is more important than running a very large number of experiments. Keep track of your experiments. Your final report should tell a coherent story about how your system evolved.

6 What to Submit

Submit a **maximum two-page PDF report** containing a link to your GitHub repository. The two-page limit does not include references. Your report should address the following.

1. Final Result

Clearly report your best test-set accuracy. Also report your validation strategy and the validation accuracy corresponding to the selected model. Include the GitHub link containing the exact code and checkpoint used to generate your final prediction.

2. Your Secret Recipe

Describe the final system concisely. Relevant details may include:

- CNN architecture;
- pretrained initialization, if any;
- input resolution;
- optimizer and learning rate;
- learning-rate schedule;
- batch size and number of epochs;
- regularization;
- data augmentation;
- loss function;
- model selection strategy;
- any additional training tricks.

Do not simply list settings. Explain which choices mattered.

3. Experimental Journey

Show evidence that your final system resulted from experimentation rather than a single training run. Include a compact table similar to the following:

Experiment	Validation Acc.	Observation
Baseline CNN	–	–
+ Augmentation	–	–
+ Training Strategy	–	–
Final Model	–	–

You do not need to report every failed experiment. Select the experiments that best explain the development of your final solution.

4. Failure Analysis

Pick at least one thing that **did not work as expected**. Explain:

- what you tried;
- why you thought it might work;
- what happened;
- what you learned from the result.

A well-analyzed failed experiment can be more valuable than another 0.5% improvement in accuracy.

5. AI + Human

Briefly discuss one example in which an AI coding assistant helped you and one example where **your judgment was necessary to improve, correct, or reject its suggestion**. Detailed documentation should be placed in `AI_USAGE.md`; only the key observation belongs in the report.

7 GitHub Requirements

All assignments must be maintained using **GitHub**. Your repository must contain:

- a clear `README.md`;
- training code;
- inference/evaluation code;
- dependency or environment information;
- instructions for reproducing your result;
- the final model checkpoint, or a link to the checkpoint;
- `AI_USAGE.md`;
- a meaningful Git commit history showing the development of the project.

Do not upload the entire project as one commit immediately before the deadline. Your code should allow me to reproduce your reported result without rewriting your program. Large checkpoints may be stored using Google Drive, Hugging Face, GitHub Releases, or another suitable hosting service. You may keep the repository private during the assignment. If it is private, make sure the instructor have access when the assignment is submitted.

8 Reproducibility

Your repository should make it possible to run commands similar to:

```
python train.py --config configs/best.yaml
python evaluate.py --checkpoint <checkpoint>
```

The exact interface is up to you. Whenever possible, specify random seeds and report the versions of important software packages. PyTorch is recommended.

9 Resources

You may use Google Colab, the Educational Cluster, or other computing resources legally available to you. If you require access to the Educational Cluster, contact me. You are responsible for complying with the policies of any external GPU or cloud resource you use. You may consult:

- research papers;
- PyTorch documentation;
- GitHub repositories;
- Stack Overflow and technical forums;
- tutorials and blogs;
- AI coding assistants.

Using external resources is part of modern research and engineering. However, **you are responsible for understanding, verifying, and adapting anything that enters your final solution.**

10 Grading

The assignment will be graded according to the following criteria:

Component	Weight
Classification performance	30%
Experimental reasoning and analysis	25%
Model/training design	20%
Reproducibility and GitHub quality	15%
Responsible and thoughtful AI-tool usage	10%

Accuracy matters, but **the highest accuracy does not automatically receive the highest grade.** A slightly lower-performing solution supported by careful experimentation and strong analysis may receive a higher grade than a poorly understood collection of tricks. A starter codebase achieving less than approximately 50% accuracy will be provided. You are expected to substantially improve upon this baseline.

11 The Leaderboard

Friendly competition is encouraged. We will maintain an informal leaderboard based on test accuracy. Recognition will be given for:

- **Best Accuracy;**
- **Best Accuracy–Efficiency Tradeoff;**
- **Most Interesting Experimental Finding.**

The leaderboard is intended to make experimentation fun; it does not replace the grading rubric.

12 Academic Integrity and Collaboration

You are encouraged to discuss general ideas with classmates, including architectures, optimization concepts, debugging strategies, and relevant papers. However, your implementation, experiments, analysis, report, and GitHub repository must be your own. Do not share code, checkpoints, detailed hyperparameter configurations, or test predictions with other students before the deadline. AI-generated code is **not exempt from this requirement**. If two students independently ask an AI system for code, each student remains responsible for adapting, understanding, testing, and integrating that code into their own solution. The instructor may ask you to explain any component of your submitted system. Inability to explain important parts of the submitted code may affect the grade.

13 Submission

The deadline is:

September 25, 2026 at 11:59 PM EDT

Submit a single PDF containing your two-page report and a working link to your GitHub repository. Before submitting, ask yourself:

*If I gave this repository to another computer vision researcher,
could they understand what I tried and reproduce my result?*

If the answer is yes, you are ready to submit.

Good luck, and have fun training your first model of the semester!